



Google Summer of Code 2024

[Metacall]

Project : BUILDER

Personal Info

Name: Ashish Kumar

Primary Email: ashishndiitr@gmail.com

Secondary Email : a_kumar1@ma.iitr.ac.in

Telephone: +91 8279296177

Time Zone: India (UTC+5.30)

GitHub: [ashpect](#)

LinkedIn: [ashish](#)

Slack/Telegram username: Ashpect

Discord Id: Ashpect#4864

University Info

University: Indian Institute of Technology, Roorkee

Major: Mathematics and Computing

Current Year: 3rd year (2026 expected graduation)

Degree: Bachelor of Science-Masters of Science (B.S-M.S) - integrated

About Me

I am a 3rd Year **Bachelor of Science** student at the **Indian Institute of Technology, Roorkee**.

I have been interested in software development since my freshman year and have been exploring this space since then. I have always loved the concept of open source and contributed to the same.

Regarding my technical experience,

I have previously worked with CNCF as a LFX Mentee under LFX.

In addition to this, I am a developer at SDSLabs which fosters and promotes technical activities on the campus. Primarily focusing on developing production level apps, It caters to all the technical needs of the students. I am also a part of the official technical student team of SPARK IITR which is the official summer internship programme of IIT Roorkee.

My hobbies include public speaking, writing and performance poetry. I love to have philosophical discussions from time to time as well. I'm always looking out for new music suggestions.

Statement of Motivation

What is your motivation for participating in Google Summer of Code?

I have always been excited by the prospect of converting ideas into products with real-world impact and was captivated by the concept of open source ever since I was first introduced to it. Since then, I have wanted to be a part of this community and have been trying to contribute in the small ways I could. GSoC provides the perfect opportunity to jump right into the Open Source World with the opportunity to work on awesome and even bigger open-source projects with experienced developers. That's why I am motivated to participate in GSoC, as it will enhance my knowledge and development skills and give me an opportunity to work on industry-scale projects under experienced mentors.

Why did you choose Metacall, and why this project idea?

One of my seniors from [SDSLabs](#), IIT Roorkee introduced me to Metacall. After reading about it, I was amazed by its possibilities of polyglot programming and function as a service, and that excites me. Also, I was amazed to know about the supportive community of Metacall and eventually realized it myself as I worked with my mentors.

Having worked on devops, cloud and image building techniques previously under CNCF, I am inherently interested in this domain. Among all the projects listed, Builder fits perfectly with my skill-set and the possibilities to learn further due to its challenging and rewarding learning experience and the skills listed : Go, Docker, Buildkit, Sandboxing, Kubernetes.

What are your expectations from us during and after the successful completion of the program?

I am looking forward to bonding with the community and making some valuable contributions to the project. I would be grateful to become a part of this community and continue my contribution beyond GSOC. I am always open to essential discussions over ideas and their implementations. I would love to receive feedback and suggestions on my contributions, as I believe it will help improve my code quality and help me become an overall better developer in the long run.

What are you hoping to learn?

I am hoping to learn working collaboratively with the community, improve my understanding of polyglot programming, buildkit, sandboxing and cross platform development and other essential skills.. Staying in continuous touch with such qualified people in the community would be an excellent opportunity for me. I am very excited to work on this project, along with the highly supportive Metacall community in the coming summers.

Previous Experience

What kind of projects have you worked on in the past? What technologies did you use?

- [Katana](#) is a work in progress distributed attack and defense CTF platform. It aims to make hosting an attack and defense CTF easy/lightweight by using concepts of **Kubernetes, Gogs, Golang, Harbor**, etc.
- [Carvel](#) : **Worked under CNCF** and successfully made imgpkg comply with the OCI Specification for image and artifact representation making it compatible with various other tools like crane, it will be further leveraged by kctrl.
- [ChainPe](#) was a team project built as a submission for **EthIndia'22**. It is a non-custodial wallet which lets you pay to any UPI ID via your cryptocurrency balance. We were the **winners** this year. I learnt a lot about **blockchain/react native**.
- [Poseidon](#) is a bot that helps us to create a discord bot without any code. Basically a discord bot to automate and customize creating a discord bot. I learnt python and how to use discord api.
- [Lib_manage](#) is a library management site written in Nodejs. Later developed it in php following mvc architecture. I learnt **Nodejs, Php** and full stack development.

Relevant Tech Stack

Programming : C/C++, Go, Javascript, Python, HTML/CSS, PHP,

DevOps/Cloud : Kubernetes, Azure K8s Services, Docker, Harbor

FullStack and misc : Git, React, Express Js with Node Js, Svelte, MongoDB, Sql, CMake

Contributions on Metacall

I was able to install metacall on macOS, build and run examples given in the doc and play around with the code. I interacted with mentors and implemented the POC for the Builder Project.

- <https://github.com/metacall/builder/pull/20> [POC : Under Review]
- <https://github.com/metacall/builder/pull/19> [Merged]
- <https://github.com/metacall/core/pull/380> [Closed with fix implemented in other PR]

Other Open Source Contributions :

- <https://github.com/sdslabs/katana/pull/72>
- <https://github.com/carvel-dev/imgpkg/pull/612>

Mentors

Fernando Vaño Garcia, Vicente Eduardo Ferrer Garcia, Gil Arasa Verge

Project Details

<https://github.com/metacall/builder>

Overview

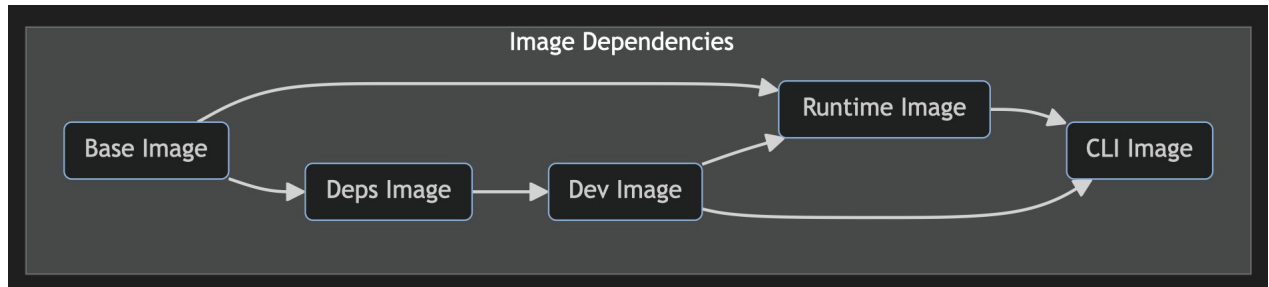
The objective of this project is to **develop a command-line interface and library** for MetaCall, enabling **selective composition of Docker images for efficient deployment** in Docker and Kubernetes environments. This involves **leveraging the Buildkit API for robust image construction**, ensuring **rootless and daemonless operation** for compatibility, and **prioritizing sandboxing for security**. The aim is to support Function-as-a-Service development by generating compact images with **reduced dependencies and attack surfaces** and making **light weight images**, facilitating streamlined deployment workflows.

The project can be divided into 4 major portions for the implementation :

1. **Efficient Docker Image Composition** : Currently, MetaCall is provided as a Docker image with all dependencies packaged together, resulting in larger image sizes. The objective is to create a CLI that allows users to selectively compose container images with MetaCall runtime, enabling them to choose only the necessary components for their applications. This selective composition should result in smaller image sizes, reducing bandwidth usage and attack surface.
2. **Utilization of Buildkit API**: Rather than templating Dockerfiles, the project aims to utilize the Buildkit API for building Docker images. This approach provides robustness and takes advantage of features like caching, improving the efficiency of the image-building process.
3. **Rootless and Daemonless Operation**: The solution must operate in a rootless and daemonless manner, ensuring compatibility with Dockerized environments, including Kubernetes clusters. This requirement allows for seamless integration into container orchestration platforms while maintaining security and permissions constraints
4. **Sandboxed Environment**: The project must prioritize security by ensuring sandboxing mechanisms are in place. This ensures that the execution environment is isolated, reducing the potential attack surface and enhancing overall security.

Efficient docker image composition using buildkit api

- Current docker images are created using dockerfiles, which do not use efficient docker image composition available in buildkit, leading to redundancy, time delays etc.
- Below is a graphical representation of the inheritance relationships between different versions of images. Bases for each image are represented by the arrows



- This multi-layer architecture dependent dockerfile structure, with each layer providing capabilities to the upper layers will help us enable the builder to flexibly merge and diff image layers.

Builder Design

Existing Images Offered By Metacall :

- Currently the base image used to build the following images is debian-slim. (You can specify the base image using the --image flag in the cli)
- Below are the images offered by metacall
 - **Dep** : It is the base of all dev images, needed for building any loader. It contains the base for any image further produced.

It contains a base image (debian-slim or any other if specified using the flag). + Clone of the metacall repo (you can specify the branch to be pulled using -b flag in cli, more about cli later) + Dependencies for dev image creation such as gcc, make, clang etc.
 - **Dev** : Dev image is built upon the base of deps image and it includes the loaders for the languages specified.
 - **Runtime** : Runtime images are for executing code based on metacall. It requires the languages' specific runtime dependencies and the loaders of the languages previously built in the dev image. Hence, you can see the copy image etc. in the docker file of runtime.

The Dockerfiles of the above said images can be found [here](#)

Issues with such approach are follows :

It leads to hardcoding and copying dependencies and loaders across images. Also the images will be needed to build again if any one of the stages are changed. Below are the ways to prevent that -

We can use efficient caching, image state management tools and [MergeOp and DiffOp](#) provided by [buildkit](#) to tackle these issues. Regarding creating the images using builder, the following approach is taken.

Please refer to the cli structure below to understand in detail, I will now directly demonstrate the cli commands as well :

Assuming -b is set to dev by default and -image to debian and ignoring other subcommands and flags for demonstration :

builder deps : contains the debian base + metacall clones + dependencies like gcc, make etc. installed.

builder deps py : same as builder deps + dependencies required to build python loaders.

builder dev : deps base + [metacall_configure("")] + metacall_build] This will contain everything except the language loaders due to the empty args provided.

builder dev py : deps base + [metacall_configure("py")] + metacall_build]. This will contain above + py loaders.

To build the final runtime image : we merge runtime base py (image state containing the runtime python dependencies) with python loaders

builder runtime py : merge(runtime base py, diff(dev base, dev base py))

builder runtime -cli py : everything in the builder runtime py + metacallcli in the runtime image

Runners as an intermediate state : Another kind of images, which are independent to the compilation of metacall/core itself are the **runners**. Those do not need development dependencies (in terms of metacall/core development) but they need end user development dependencies. For example, in Python, they will need python3 and pip3 in order to install dependencies and run python code, but it won't need python3-dev package, instead it will be enough with python3-lib which is already included by python3. This is needed in order to have a minimal image in which to run installation scripts (ci/cd) of the end user projects. They do not need to inherit from dev-deps-base (usually). There's some exceptions, like for example, sometimes NodeJS has requirements like electron which need a compiler in order to install them, similar can happen in python although I haven't seen that in the same frequency.

Those images will be used as intermediate images in order to build the final image of the end user. They will execute pip install ... or npm install ... during "end-user build time" (i.e during when we are creating the end user images, not the images of metacall/builder itself).

After the first poc approach of the final image is completed avoiding runners, we can look into the feasibility of adding runners by adding an intermediate phase where runners are executed, and the dependencies of the user are copied from the runners instead. This is subject to discussion with the mentor after the poc completion.

CLI Options

The Cli is aimed to be intuitive and user friendly. It can be divided as follows :
Consider this demo placeholder command.

```
builder [command] [globalFlags] [imageSpecification] [subFlags]  
[languages as Argument]
```

- Image specifications subcommand include deps, dev, runtime etc. (Image types) required to initiate the cli to form the required image.
- Global flags are flags which can be set for every image and are compatible with it, such as -b (branch), required to allow users to specify the branch of metacall from github to be pulled.
- Another example of a global flag is the -image flag which specifies the base image, which is debian-slim by default.
- Sub Flags include image specific flags , for example, in the runtime image, the user can choose whether to have the cli installed and can specify it with the -cli flag.
- Languages as arguments : Finally the short form of languages can be given as an input, multiple in one line, such as rb, py, c (ruby, python, c etc.) The whole list will be available in the -help subcommand along with a fellow example.
- Languages provided are mapped to the list of languages available which are used to install the loaders and dependencies for only the language.

An example command may look like this :

```
builder -b development -image ubuntu:latest runtime -cli py c rb
```

Testing

To test the images produced, we require testing and validation across different deployment scenarios to ensure reliability, performance and compatibility. To approach it, we can do it in following ways

Unit Testing : At each llb.State(builtkit image state represented as lower level build) , we can attach tests for each step's unit testing.

Pros : Exact point of failure recognition.

Cons : Too many steps/state updation as we keep improving the builder will lead to implementing tests for each of them.

Script Testing : We can create a script for checking the final docker images directly by running them in a light base container.

Pros : Single script with multiple arguments designed for testing could be implemented. This will have implication and backward compatibility to all the images currently build with docker as well, and aid to testing them as well.

Cons : Error handling needs to be specified and mapped to the state where llb changed in some way. A good way could be to mark the llb states in sequential order throughout and map it with the errors thrown by the script to help the developer in builder easily identify the point of failure.

Using Dive in the CI : Dive is an image analysis tool which can be used to check for the images diff that the builder worked correctly as well as retrieve the stats of the images for later purposes. Mentioned in [this](#) issue.

Dockerizing Builder

To dockerize the builder, we will need to run buildkit inside docker without the root privileges (rootless and daemonless).

First we will install rootless kit and buildkit inside the docker container. In the case of Arm containers, we can use [limactl](#) to create lightweight containers and run buildkit rootless. This is subject to discussion based on a lightweight approach if found. The implementation of both can be found in the builder script inside the repo.

Writing a multi-stage build dockerfile, our aim will be to get a lightweight image with the builder executable inside it.

To create images inside the container from the llb's, we can use the buildkit itself. In case of running issues with buildkit, we can also leverage the use of Buildah or Kaniko as the image building tools inside the containers. Kaniko doesn't depend on a docker daemon and executes each command within a Dockerfile or equivalent commands completely in userspace, while not requiring root privileges catering to our need.

Worst case, if the need arises to run "root requiring" workloads such as docker or etc, we can use [sysbox](#), a handy tool for running such workloads empowering rootless containers.

Further Goals

Further goals include the following :

- Adding more user friendly commands in the cli such as up (to set up buildkit etc) instead of script dependent installation
- Making UI better with error logging with color codings, showing visual installation features, etc.
- Setting up a CI similar to one in [core](#) for updating the images offered by metacall on docker hub each time a new version of the distributable is made.

Project Timeline

Pre GSoC Period	
April 2th - May 1st	• I would continue working on my current PRs and build a more robust poc for the same. • Understand more about polyglot programming and legacy/history of metacall and how it works under the hood.
Community Bonding Period	
May 1st - May 26th	• Get to know more about the Metacall community. • Discussion with my mentor to fix any shortcomings in the proposal and discuss better ideas while coding the PoC.
Coding Period	
May 27th - June 3rd	• Create a testing framework and test the poc with the runtime, dev and deps images. • Using and getting familiar with Dive. • Writing scripts for testing core/tools/dockerfiles
June 3rd - June 10th	• Complete the builder for runtime, dev and deps images and publish it, get reviewed by the mentor.
June 10th - June 17th	• Improving the Cil with better user experience, adding error logging, color coding, better visualization

June 17th - June 24th	Buffer time to complete any additional tasks(if any)
June 24th - July 8th	• Implement copying deps using runners as an intermediate step • Packaging the builder and any/all small improvements
July 8th - July 12th	Mid Evaluation
July 12th - July 22th	• Dockerize the Builder as per the above explanation. • Test with kubernetes cluster for rootless pods, push and pull from private registry
July 22th - July 31st	• Improve code for sandboxing demands, isolate dependencies, limit permissions, etc. • Add monitoring and logging mechanisms
August 1st - August 12th	• Discuss and work on creating the CI to update the images and test and update all the docker hub images
12th Aug - 19rd Aug	• Reviews, Improvements and final submissions.
28th Aug - 4th Sep	Final Phase

Availability

I would be able to dedicate around 30 hours a week and 40-45+ hours during my summer vacations (15th May to 17th July). I generally work from 1 PM to 3 AM IST, although I find it flexible to adjust my working hours if required.

Other than this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

Why me?

I've been quite enthusiastic and eager to learn new things and try my hand at new technologies. I've already worked on a couple of projects like Katana, Rootex and a DevOps intern CNCF. Hence, I can apply the same knowledge and experience drawn from there. I am confident that I can contribute at par to the standards an open-source organization demands.

Apart from this, I would get to work with many professionals as my mentors. This will be a huge learning experience for me. Having worked in numerous teams and open source projects, I am aware of the software development process and the best practices and recommendations.

Involvement after GSoC

I am learning a lot by contributing to Metacall. Even after the GSoC period ends, I plan on contributing to this organization by adding to my past projects and working on open issues because of the familiarity of the technical stack and the new challenges that I am continually offered in the process.

Also, I would like to complete the secondary goals mentioned in my proposal as well as help newcomers in the community to grow alongside in the spirit of open source.

References

- <https://core.metacall.io/#/docs/contribute>
- <https://github.com/metacall/core>
- <https://github.com/metacall/builder>
- <https://github.com/moby/buildkit/tree/master>
- <https://github.com/wagoodman/dive>